VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Deep Learning and Its Applications (CO3133)

Assignment 1

Foundations of Deep Learning Pipelines and Architectures

Topic: From Linear Models to Modern Sequence Models:
A Comparative Study for Image Classification.

Instructor: Lê Thành Sách

Class: CC01

Group: G-10

Semester: 1 – Year 2026–2027

Students:

Trịnh Lê Minh	2352765
Trần Phước Sang	2353044
Đặng Trần Thái Bảo	2452120

HO CHI MINH CITY, OCTOBER 2026

Contents

Workload Distribution 3

I Problem and Data Description 4

1 Problem Formulation 4

2 Dataset Description 4

3 Exploratory Data Analysis 6
3.1 Core EDA . . . 6
3.2 Task-Specific EDA (Classification) . . . 8

4 Data Splitting 9
4.1 Train/Val/Test Split Strategy . . . 9
4.2 Leakage Awareness & Mitigation . . . 9

5 Preprocessing 10

II Methodology 11

6 Pipeline Overview 11

7 Baseline and Main Methods 12

8 Training Setup 16

9 Experimental Design 17

III Implementation Results 18

10 Training Behavior 18

11 Quantitative Results 20

12 Qualitative Results 21

13 Comparison and Discussion 24

14 Error Analysis 24

15 Limitations and Conclusion 25

AI Usage Disclosure 26

List of Figures

1 Representative Fashion-MNIST examples selected reproducibly with seed 42. . . . 5
2 Fashion-MNIST training-pool class distribution. . . . 7
3 Reproducibly selected Fashion-MNIST training examples. . . . 7
4 PCA, t-SNE, and UMAP projections of sampled Fashion-MNIST images. . . . 8
5 Cosine similarity between class-level mean images. . . . 9

6 End-to-end Fashion-MNIST pipeline shared by the five implemented architectures. 12
7 Linear classifier architecture. 13
8 MLP architecture with nonlinear hidden layers and dropout. 13
9 CNN spatial feature extractor and dense classifier. 14
10 GRU row-sequence representation, recurrent hidden state, and classifier. 15
11 Vision Transformer patch projection, class and position tokens, encoder stack, and classifier. 15
12 Linear training and validation curves. Validation loss plateaus early, indicating limited model capacity. . . 18
13 MLP training and validation curves. Nonlinear capacity improves the validation trajectory over Linear. . . 19
14 CNN training and validation curves. The lowest validation loss, 0.1702, occurs at epoch 55. 19
15 GRU training and validation curves. The selected epoch-53 checkpoint has validation loss 0.2842. 20
16 ViT training and validation curves. The minimum validation loss, 0.3062, occurs at epoch 57; training stopped after epoch 67. 20
17 Qualitative test predictions from Linear, MLP, CNN, GRU, and ViT. Each example shows the true class and each model's predicted class. 22
18 Test confusion matrices: Linear, MLP, CNN, GRU, and ViT. 23

List of Tables

1 Workload allocation among team members. 3
2 Dataset summary. 5
3 Official training-pool class distribution. 6
4 EDA statistics. 8
5 Data split configuration. 9
6 Preprocessing pipeline. 10
7 Implemented methods and trainable parameter counts. 13
8 Training configuration for the five implemented model families. 16
9 Experimental design for the model-family comparison. 17
10 Predictive performance on the Fashion-MNIST test set. 21
11 Resource and efficiency measurements. 21
12 Major error groups across all models. Counts are errors within the indicated true-class subset. 24

Workload Distribution

No	Name	Student's ID	Work	Contribution
1	Trịnh Lê Minh	2352765	Main Develop	100%
2	Trần Phước Sang	2353044	Verify Dev and Outline Report	100%
3	Đặng Trần Thái Bảo	2452120	Complete Report and Slides	100%

Table 1: Workload allocation among team members.

Part I

Problem and Data Description

1 Problem Formulation

Motivation. Fashion-MNIST is a controlled image-classification benchmark that is more visually demanding than digit recognition while remaining small enough for systematic architecture comparison. The assignment investigates how linear, dense nonlinear, spatial convolutional, recurrent, and self-attention architectures affect predictive quality and computational cost under one shared pipeline.

Input and prediction unit. Each prediction concerns one grayscale clothing image $x \in \mathbb{R}^{1\times 28\times 28}$. Linear and MLP models flatten the image to 784 values; CNN preserves the two-dimensional grid; GRU reads the image as 28 ordered row vectors with 28 features per timestep; and ViT projects the image into 49 non-overlapping patches and adds a class token. Pixel values are first scaled to $[0, 1]$ and then normalized using statistics from the training split.

Output. The target $y \in \{0, \ldots, 9\}$ is one of ten Fashion-MNIST classes. Every model returns ten unnormalized logits, and the predicted class is the index of the largest logit. No softmax is applied before `CrossEntropyLoss`.

Mathematical formulation.

$$\hat{y}_i = \arg\max_{c\in\{0,\ldots,9\}} f_\theta(x_i)_c, \qquad \theta^* = \arg\min_\theta \frac{1}{N}\sum_{i=1}^{N} \mathcal{L}_{\mathrm{CE}}(f_\theta(x_i), y_i)\,. \tag{1}$$

The research question is: *How do representation choice and architectural inductive bias change Fashion-MNIST accuracy, macro-F1, model size, training cost, and class-level errors when the data split and evaluation protocol are fixed?* The comparison evaluates Linear, MLP, CNN, GRU, and ViT using the same logged run protocol.

Main difficulties. Shirt, T-shirt/top, Pullover, and Coat have overlapping low-resolution silhouettes. Flattening discards explicit spatial neighborhoods, whereas row-wise recurrent processing introduces an ordering that is not fully natural for a two-dimensional image. CNN supplies a strong local image prior, while ViT must learn spatial relations from patch and position embeddings. The experiment must therefore interpret predictive quality alongside capacity and computational cost.

2 Dataset Description

Fashion-MNIST is a dataset of Zalando clothing images intended as a more challenging alternative to handwritten-digit benchmarks. It is distributed through the `torchvision.datasets.FashionMNIST` interface. The project is released under the MIT license.

Table 2: Dataset summary.

Property	Description
Source and interface	Fashion-MNIST through torchvision 0.29.0.
Collection and labeling	Centered 28×28 grayscale product images with one integer class label per image.
Number of samples	60,000 official training images and 10,000 official test images.
Classes	T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, and Ankle boot.
Storage and formats	IDX image/label files plus gzip archives under `Assignment1/data/FashionMNIST/raw`; approximately 82 MB including compressed and extracted copies.
Biases and limitations	Small centered grayscale objects, limited background variation, low resolution, and substantial visual overlap among upper-body garments.

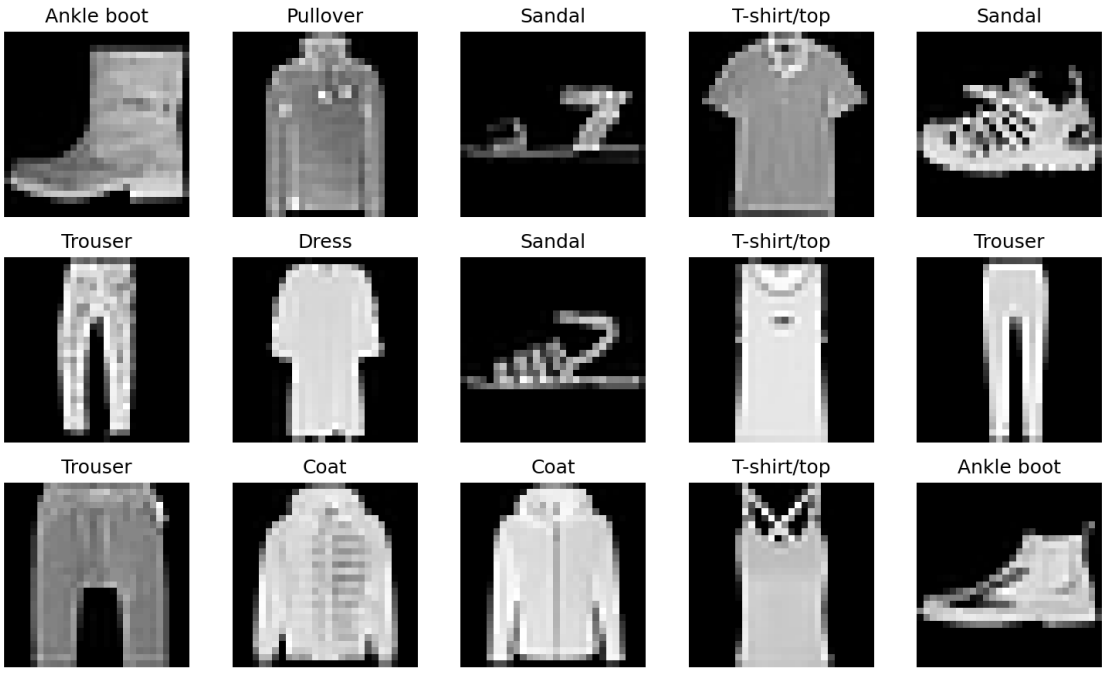


Figure 1: Representative Fashion-MNIST examples selected reproducibly with seed 42.

Table 3: Official training-pool class distribution.

Class / target	Count	Percentage
T-shirt/top	6,000	10%
Trouser	6,000	10%
Pullover	6,000	10%
Dress	6,000	10%
Coat	6,000	10%
Sandal	6,000	10%
Shirt	6,000	10%
Sneaker	6,000	10%
Bag	6,000	10%
Ankle boot	6,000	10%
Total	60,000	100%

The classes are perfectly balanced in the official training pool. Stratified splitting therefore preserves 5,400 training and 600 validation examples per class, so accuracy is meaningful and macro-F1 remains useful for exposing class-specific weaknesses rather than correcting a frequency imbalance.

3 Exploratory Data Analysis

3.1 Core EDA

The core EDA describes the dataset size, tensor shape, pixel range, pixel statistics, class distribution, and representative images. All random examples are selected reproducibly using seed 42.

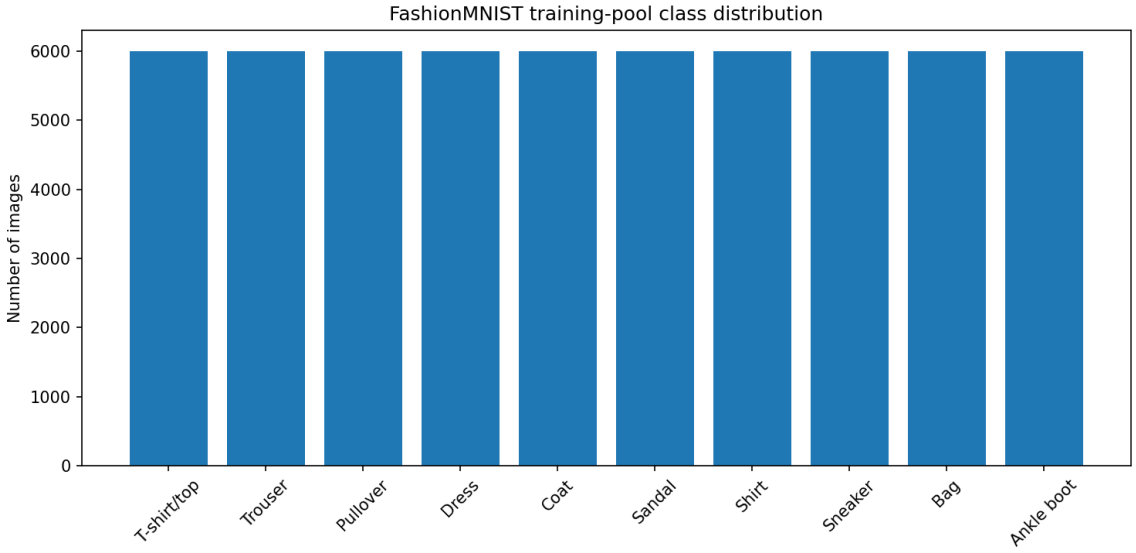


Figure 2: Fashion-MNIST training-pool class distribution.

The class distribution is perfectly balanced, with 6,000 images for each of the ten classes in the official training pool. This balance reduces the risk that the classifiers are dominated by a majority class.

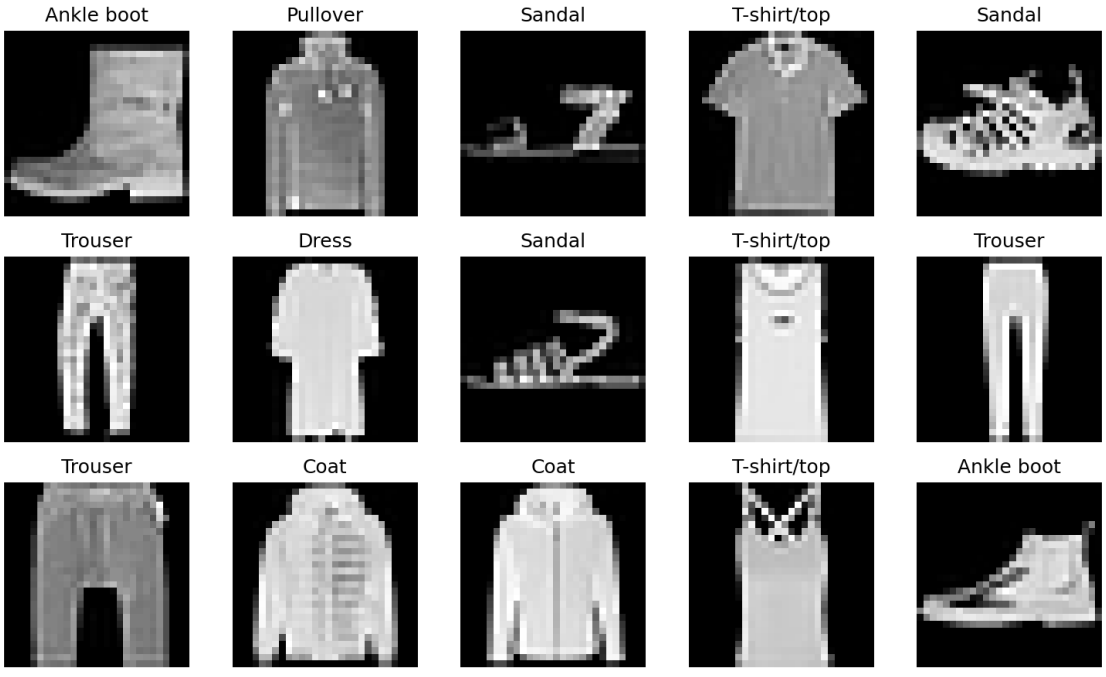


Figure 3: Reproducibly selected Fashion-MNIST training examples.

The sampled images confirm that the dataset contains centered grayscale clothing objects with visually meaningful class labels. They also illustrate within-class variation in pose, shape, texture, and intensity, motivating comparison of flattened-pixel, spatial, and row-sequence representations.

Table 4: EDA statistics.

Statistic	Value	Notes
Mean	0.2860	Computed on training-pool pixels scaled to $[0, 1]$.
Standard deviation	0.3530	Computed on training-pool pixels scaled to $[0, 1]$.
Minimum / maximum	0.0 / 1.0	Raw pixel values after scaling.
Missing or invalid values	0 detected	No missing-value handling was required.

The images are represented as $1 \times 28 \times 28$ tensors with pixel values in $[0, 1]$ before normalization. The measured mean and standard deviation are consistent with the values used by the data-loading pipeline; the implementation recomputes them from the 54,000-image training subset to prevent leakage.

3.2 Task-Specific EDA (Classification)

For the classification task, the EDA additionally examines the geometry of the image representations and the similarity between class-level average images.

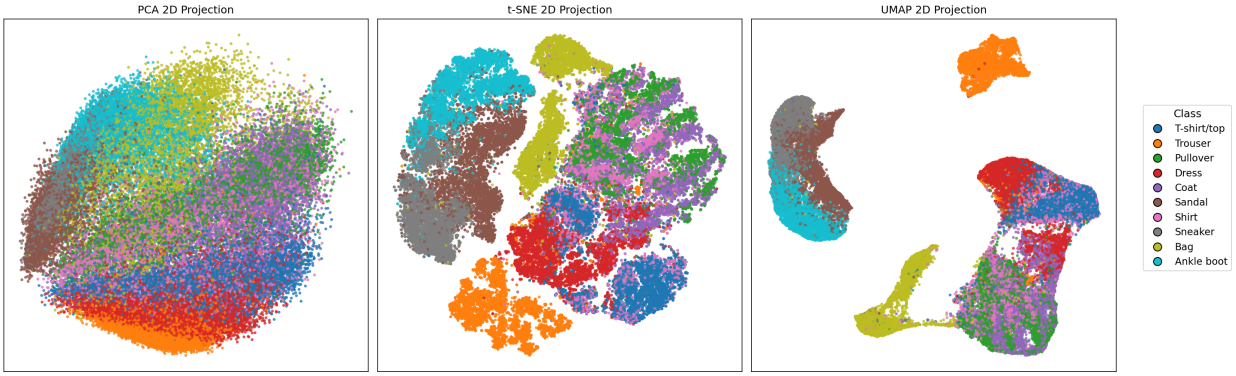


Figure 4: PCA, t-SNE, and UMAP projections of sampled Fashion-MNIST images.

The dimensionality-reduction views show that some classes form relatively coherent groups, while visually similar clothing categories overlap. PCA provides a broad linear view, whereas t-SNE and UMAP reveal more localized nonlinear structure. These projections suggest that footwear classes are often easier to separate, while upper-body categories may require more expressive decision boundaries.

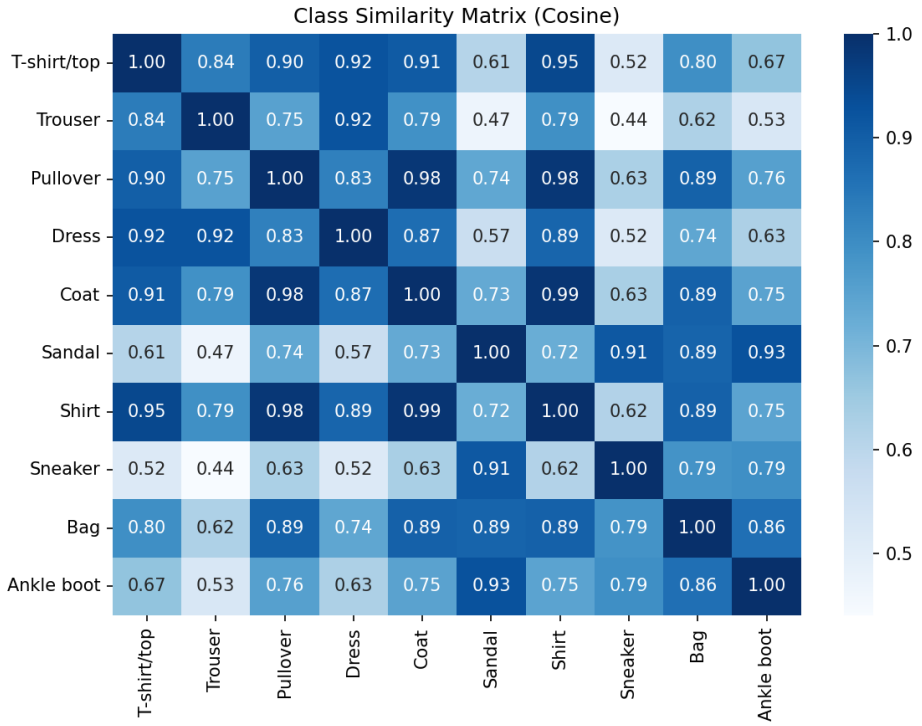


Figure 5: Cosine similarity between class-level mean images.

The cosine-similarity matrix highlights pairs of classes with similar average visual structure. For example, Pullover–Coat and Coat–Shirt have high similarity, while Sandal–Trouser and Sneaker–Trouser are less similar. The high similarity among several upper-body classes indicates likely confusion regions for classification and motivates the later error analysis.

4 Data Splitting

4.1 Train/Val/Test Split Strategy

The official 60,000-image training pool is split into 54,000 training images and 6,000 validation images using a stratified 90/10 split with random seed 42. The official 10,000-image test set is kept separate and is used only for final evaluation. Splitting is performed at image level before training, with no overlap between partitions.

Table 5: Data split configuration.

Split	Samples	Percentage	Selection rule
Train	54,000	90% of official train pool	Stratified custom split, seed 42
Validation	6,000	10% of official train pool	Stratified custom split, seed 42
Test	10,000	100% of official test set	Official Fashion-MNIST test partition

4.2 Leakage Awareness & Mitigation

The data pipeline applies the following leakage-prevention checklist:

- **Split before fitting preprocessing:** the training and validation indices are created before any normalization statistics are estimated.
- **Training-only feature scaling:** the mean and standard deviation are computed from the 54,000 training images only. These statistics are then reused unchanged for validation and test inputs.
- **Validation isolation:** validation images are used for model selection and early stopping, but their pixel values do not affect normalization parameters or gradient updates.
- **Test isolation:** the official test set is accessed only for final evaluation and is not used for training, hyperparameter selection, or preprocessing fitting.
- **No sample overlap:** train, validation, and test partitions are disjoint at the image level.
- **Train-only augmentation:** random flips and affine transformations are applied only to training images; validation and test inputs use deterministic transforms.

This procedure follows the principle of *Feature Scaling Leakage Prevention*: preprocessing parameters must be learned from the training set only, then applied to all other partitions without refitting.

5 Preprocessing

Images are converted to tensors and normalized using statistics calculated from the 54,000-image training split. Training images additionally receive random horizontal flips and small affine transformations; validation and test images use deterministic preprocessing only.

Table 6: Preprocessing pipeline.

Stage	Operation	Configuration
Cleaning	No additional cleaning	Images and labels are read through torchvision; no missing or invalid records were detected.
Tensor conversion	`ToTensor()`	Converts `uint8` pixels from $[0, 255]$ to floating point in $[0, 1]$ with shape $[B, 1, 28, 28]$.
Normalization	$(x - \mu_{\text{train}})/\sigma_{\text{train}}$	Training-split statistics are reused unchanged for validation and test data.
Augmentation	Horizontal flip and affine transform	Training only: flip probability 0.5; rotation $[-10, 10]^\circ$; translation 0.1; scale $[0.9, 1.1]$.
Linear / MLP input	Flatten internally	$[B, 1, 28, 28] \rightarrow [B, 784]$.
CNN input	Preserve spatial grid	$[B, 1, 28, 28]$.
GRU input	Remove channel axis	$[B, 1, 28, 28] \rightarrow [B, 28, 28]$: 28 row timesteps with 28 features each.
ViT input	Non-overlapping patch projection	$[B, 1, 28, 28] \rightarrow [B, 49, 128]$: 49 patches of size 4×4, each embedded in 128 dimensions; a class token makes 50 tokens.

Augmentation is intentionally excluded from validation and test data. Consequently, validation accuracy may exceed training accuracy because validation uses clean inputs and disables dropout, while training metrics are measured on augmented inputs with dropout active.

Part II

Methodology

6 Pipeline Overview

All five implemented architectures use the same split, preprocessing objects, optimizer family, trainer, checkpoint rule, and test evaluator. Figure 6 shows the complete execution path. Training batches are augmented and normalized; validation and test batches are normalized without augmentation. At every epoch, the trainer records loss and accuracy, saves a checkpoint only when validation loss improves by at least 10^{-4}, and stops after ten consecutive non-improving epochs. The best-validation-loss checkpoint is restored before the official test set is evaluated. The captured log contains complete test metrics and resource measurements for all five models.

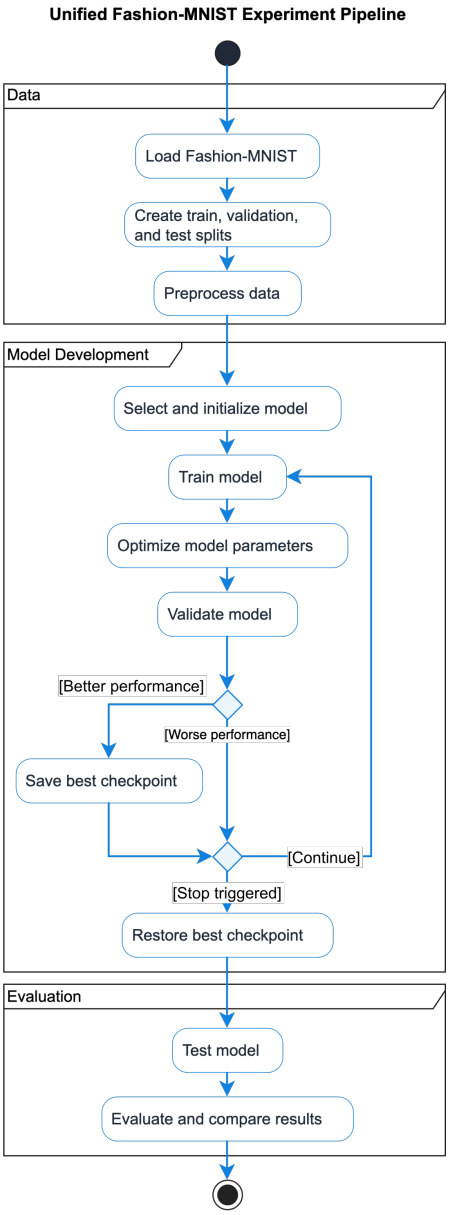


Figure 6: End-to-end Fashion-MNIST pipeline shared by the five implemented architectures.

The implementation separates responsibilities across `dataloader.py` (splits and transforms), `models.py` (architectures), `trainer.py` (training, validation, early stopping, and checkpoints), `metrics.py` (predictive and resource metrics), and `plotter.py` (EDA and result figures). This separation allows the model family to change without changing the evaluation protocol.

7 Baseline and Main Methods

All five architectures and their forward-pass wiring were designed for this assignment using PyTorch modules; they are not pretrained models. The convolutional, linear, recurrent, attention, activation, pooling, normalization, and dropout operations use PyTorch's library layers. Models receive a batch of normalized grayscale images with shape $[B, 1, 28, 28]$

and return ten raw class scores (logits) per image. The code snippets below show each implemented forward pass; each model's diagram, code, and discussion are grouped together for easier comparison.

Table 7: Implemented methods and trainable parameter counts.

Method	Input representation	Architecture	Main inductive bias	Parameters
Linear	Flattened $[B, 784]$	$784 \to 10$	One global linear boundary	7,850
MLP	Flattened $[B, 784]$	$784 \to 512 \to 256 \to 128 \to 10$	Nonlinear global feature interactions	567,434
CNN	Spatial $[B, 1, 28, 28]$	Two Conv–ReLU–Pool blocks; dense classifier	Locality and shared spatial filters	1,691,914
GRU	Row sequence $[B, 28, 28]$	One GRU layer ($H = 128$); final-state classifier	Ordered sequential dependence	61,962
ViT	49 patch tokens $[B, 49, 128]$ plus class token	4-layer Transformer; 4 heads; $d = 128$	Global token-to-token attention with learned position information	803,338

Linear / softmax baseline

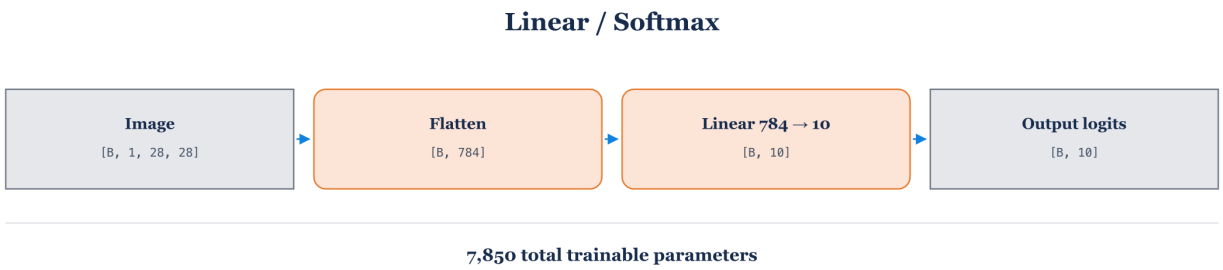


Figure 7: Linear classifier architecture.

Forward function. The code flattens each image while preserving the batch dimension, then passes the resulting 784-value vector through the linear layer. The result contains ten logits per image.

```
def forward(self, images):
    vectors = torch.flatten(images, start_dim=1, end_dim=-1)
    return self.fc(vectors)
```

The returned values are logits; softmax is used only when class probabilities are needed, since cross-entropy loss consumes logits directly.

Discussion. This is the simplest baseline: each output class is a weighted sum of all 784 pixels. It is fast and provides a useful reference point, but has no hidden nonlinear transformations and does not explicitly represent local spatial patterns.

Multilayer perceptron

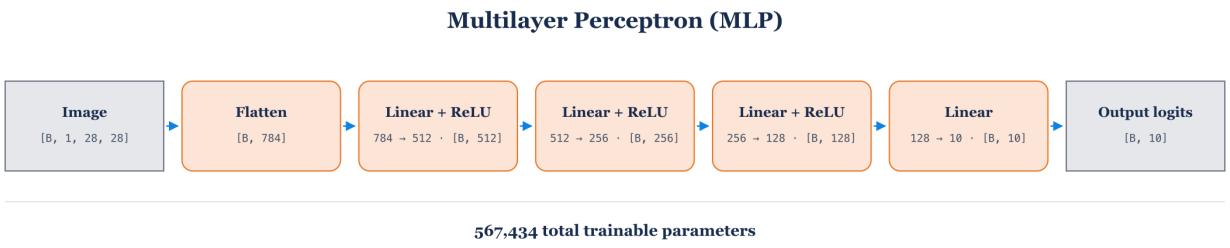


Figure 8: MLP architecture with nonlinear hidden layers and dropout.

Forward function. The input is flattened, then passed through three linear–ReLU–dropout stages and a final linear layer that returns ten logits.

```python
def forward(self, images):
    vectors = torch.flatten(images, start_dim=1, end_dim=-1)

    out = self.fc1(vectors)
    out = self.relu1(out)
    out = self.drop1(out)

    out = self.fc2(out)
    out = self.relu2(out)
    out = self.drop2(out)

    out = self.fc3(out)
    out = self.relu3(out)
    out = self.drop3(out)

    out = self.fc4(out)
    return out
```

Discussion. The three hidden layers allow nonlinear combinations of pixels and have substantially more representational capacity than the linear baseline. Flattening the image before processing, however, means that neighboring pixels are not treated specially and the model does not preserve a two-dimensional feature map.

Convolutional neural network

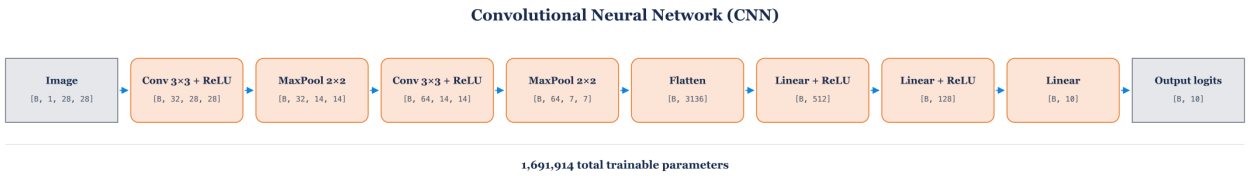


Figure 9: CNN spatial feature extractor and dense classifier.

Forward function. The feature extractor applies two convolutional blocks (convolution, ReLU, max pooling). Its output is passed to the dense classifier, which returns ten logits.

```python
def forward(self, images):
    features = self.features(images)
    logits = self.classifier(features)
    return logits
```

The operations inside `self.features` and `self.classifier` are the layers defined in the model constructor.

Discussion. Convolutions learn local filters and reuse the same weights across image locations; pooling reduces spatial resolution while retaining strong responses. These properties make the CNN well matched to image structure. Its dense classifier then combines the extracted spatial features to predict a clothing class.

Gated recurrent unit

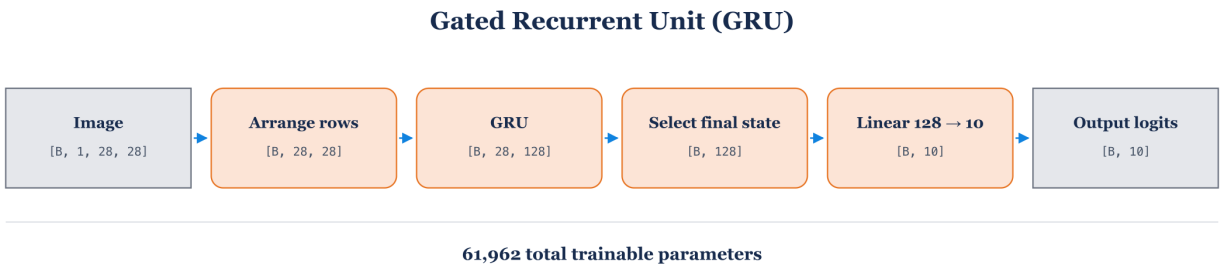


Figure 10: GRU row-sequence representation, recurrent hidden state, and classifier.

Forward function. The channel dimension is removed to represent the 28 image rows as a sequence. The GRU processes that sequence; its final hidden state is passed through dropout and the classifier.

```
def forward(self, images):
    sequences = images.squeeze(1)
    _, hidden = self.gru(sequences)

    final_hidden = self.dropout(hidden[-1, :, :])
    logits = self.classifier(final_hidden)
    return logits
```

The recurrent stage has $3HI + 3H^2 + 6H = 60{,}672$ parameters for input size $I = 28$ and hidden size $H = 128$; the classifier adds 1,290 parameters.

Discussion. The GRU can carry information from earlier rows to later rows, allowing the final state to summarize the image as a sequence. This imposes a top-to-bottom ordering on a two-dimensional object and does not retain local two-dimensional structure as directly as a CNN.

Vision Transformer

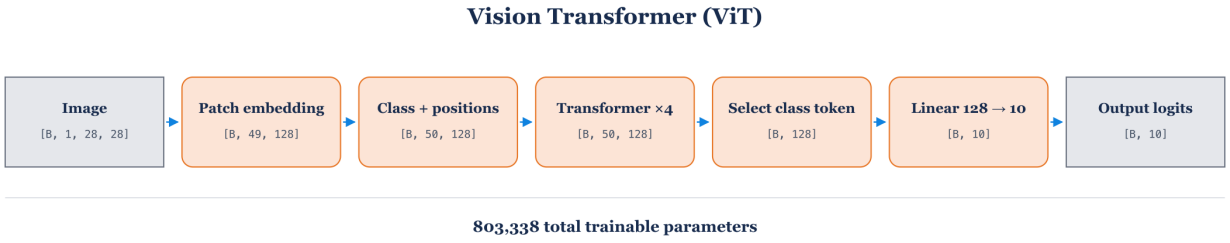


Figure 11: Vision Transformer patch projection, class and position tokens, encoder stack, and classifier.

Forward function. The convolution creates patch embeddings. The forward pass then prepends the class token, adds position embeddings, encodes the tokens, and classifies the encoded class token.

```
def forward(self, images):
    patches = self.patch_embedding(images).flatten(2).transpose(1, 2)
    class_tokens = self.class_token.expand(images.shape[0], -1, -1)
```

```
        tokens = torch.cat((class_tokens, patches), dim=1)
        tokens = self.embedding_dropout(tokens + self.position_embedding)
        encoded = self.encoder(tokens)
        image_representation = self.norm(encoded[:, 0])
        return self.classifier(image_representation)
```

Here `encoded[:, 0]` selects the encoded class token for each image in the batch. Each encoder layer uses a 512-dimensional feed-forward block with GELU activation and dropout $p = 0.2$. The model has 803,338 trainable parameters: 2,176 in the patch projection, 6,528 in the class and position embeddings, 793,088 in the encoder, 256 in final normalization, and 1,290 in the classifier.

Discussion. Self-attention lets each patch representation combine information from all other patches, while learned position embeddings provide information about where each patch came from. The class token serves as a learned summary passed to the classifier. In the recorded run, ViT reaches 87.93% test accuracy and 0.8771 macro-F1, below the CNN and GRU. Its 0.265 ms per-image inference time is the slowest measured, consistent with the cost of attention over 50 tokens.

8 Training Setup

Table 8: Training configuration for the five implemented model families.

Setting	Value
Framework	Python 3.13 runtime; PyTorch and torchvision versions were not recorded in the captured run log.
Optimizer	AdamW.
Learning rate / weight decay	0.001 / 0.1.
Batch size / workers	256 / 1 worker.
Maximum epochs	200.
Scheduler	None.
Regularization	Training augmentation for all models; dropout 0.2 in MLP, CNN, GRU, and ViT.
Early stopping	Stop after ten consecutive epochs without validation-loss improvement of at least 10^{-4}.
Checkpoint criterion	Lowest validation loss; restore the selected checkpoint before testing.
Randomness	Seed 42 for Python, NumPy, PyTorch, splitting, and DataLoader shuffling; deterministic PyTorch algorithms enabled.
Hardware	CPU-only execution; CUDA was unavailable. The exact CPU model and full session identity were not recorded.
Mixed precision	Not used.
Train time	Linear 762.21 s; MLP 1,130.64 s; CNN 2,025.71 s; GRU 1,316.61 s; ViT 7,795.84 s.
Hyperparameter selection	One fixed configuration from `config.py`; no grid or Bayesian search.

The ViT run used the same AdamW, learning rate, weight decay, augmentation, split, seed, and checkpoint rule as the other models. Because the exact processor was not logged, recorded training and inference times are observed measurements, not hardware-normalized benchmarks.

9 Experimental Design

The comparison changes the architecture and its input representation while holding the data and evaluation pipeline fixed. Each hypothesis is evaluated on the official 10,000-image test set only after selection by validation loss.

Table 9: Experimental design for the model-family comparison.

Experiment	Hypothesis	Changed factor	Fixed factors	Conclusion criteria
EXP-01 Linear	A linear classifier provides a low-cost reference but underfits overlapping clothing classes.	Single affine mapping on 784 flattened pixels.	Split, seed, transforms, optimizer, loss, checkpointing, and metrics.	Establish the minimum accuracy/F1 and resource cost.
EXP-02 MLP	Nonlinear dense layers improve on Linear despite losing spatial layout.	Three ReLU hidden layers and dropout.	Same shared protocol.	Higher accuracy and macro-F1 than Linear, interpreted with size/time cost.
EXP-03 CNN	Local filters and weight sharing give the best image-classification performance.	Spatial convolutions, pooling, and dense head.	Same shared protocol.	Highest accuracy/F1 and fewer upper-body errors, with resource trade-off reported.
EXP-04 GRU	Row-sequence recurrence beats flattened dense models with fewer parameters but remains below CNN.	28 row timesteps and 128-dimensional recurrent state.	Same shared protocol.	Accuracy/F1 above MLP, parameter count below MLP, and timing compared explicitly.
EXP-05 ViT	Patch-level global attention may model long-range image interactions, but may not outperform CNN on this small dataset without a strong locality prior.	4×4 patches, learned class and position embeddings, and a four-layer Transformer encoder.	Same shared protocol.	Accuracy slightly below GRU, but the training time is much longer compared to the rest models.

The primary metric is test accuracy; macro-F1 is the secondary predictive metric. Parameter count, serialized model size, training time, inference time, learning curves, confusion matrices, and selected qualitative examples provide efficiency and behavioral evidence. No conclusion is based on accuracy alone.

Part III

Implementation Results

10 Training Behavior

The trainer records loss and accuracy after every epoch, selects the lowest validation-loss checkpoint, restores it, and evaluates the test set once. Linear stopped after epoch 30 and restored epoch 20; MLP stopped after epoch 41 and restored epoch 31; CNN stopped after epoch 65 and restored epoch 55; GRU stopped after epoch 63 and restored epoch 53; ViT stopped after epoch 67 and restored epoch 57 (validation loss 0.3062).

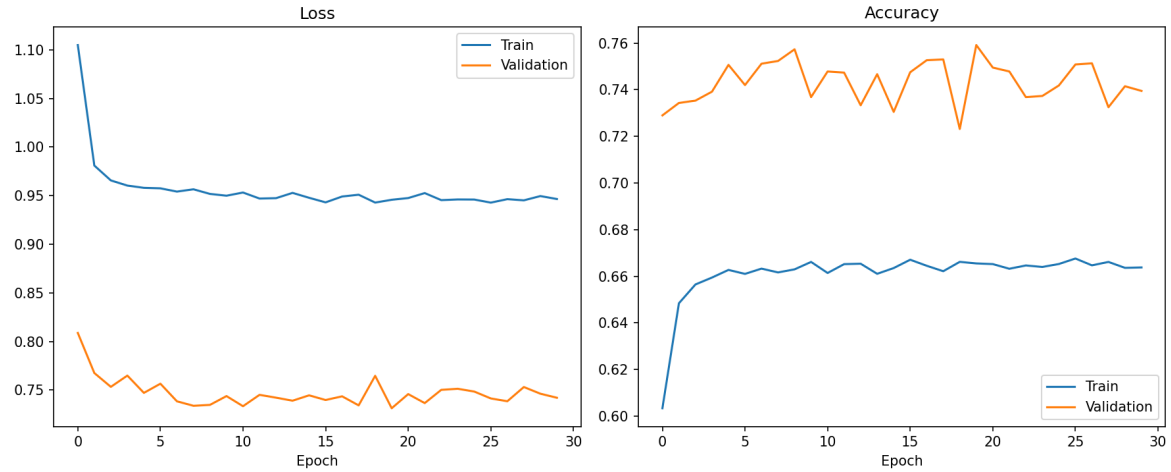


Figure 12: Linear training and validation curves. Validation loss plateaus early, indicating limited model capacity.

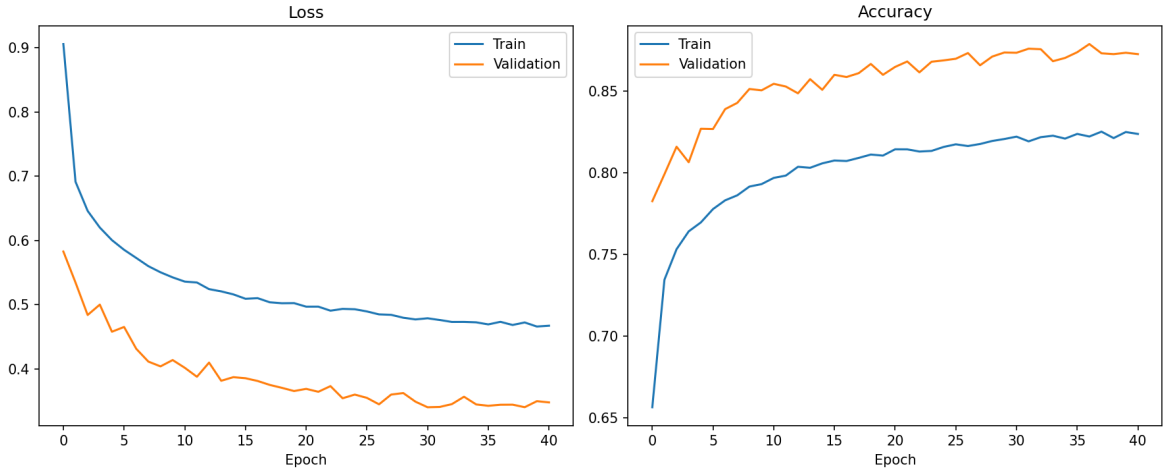


Figure 13: MLP training and validation curves. Nonlinear capacity improves the validation trajectory over Linear.

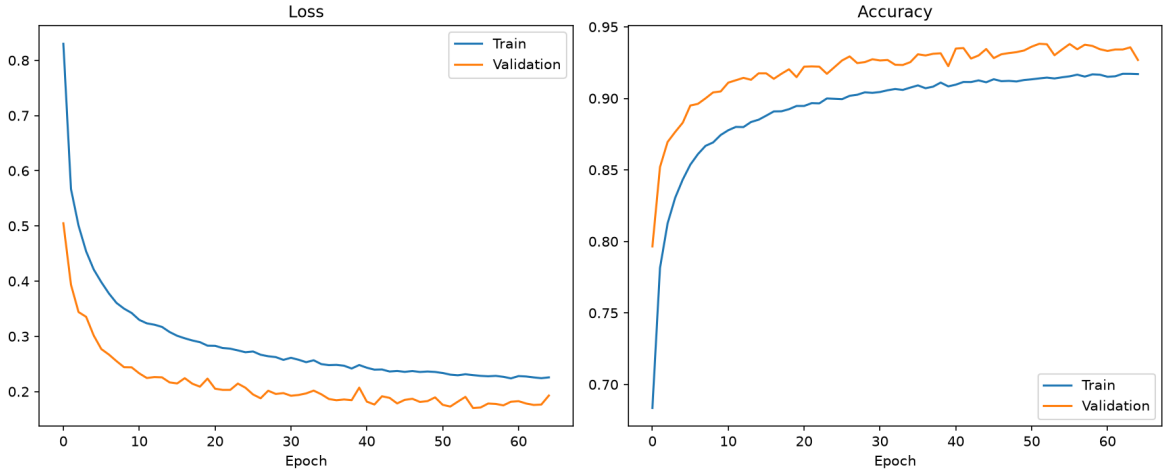


Figure 14: CNN training and validation curves. The lowest validation loss, 0.1702, occurs at epoch 55.

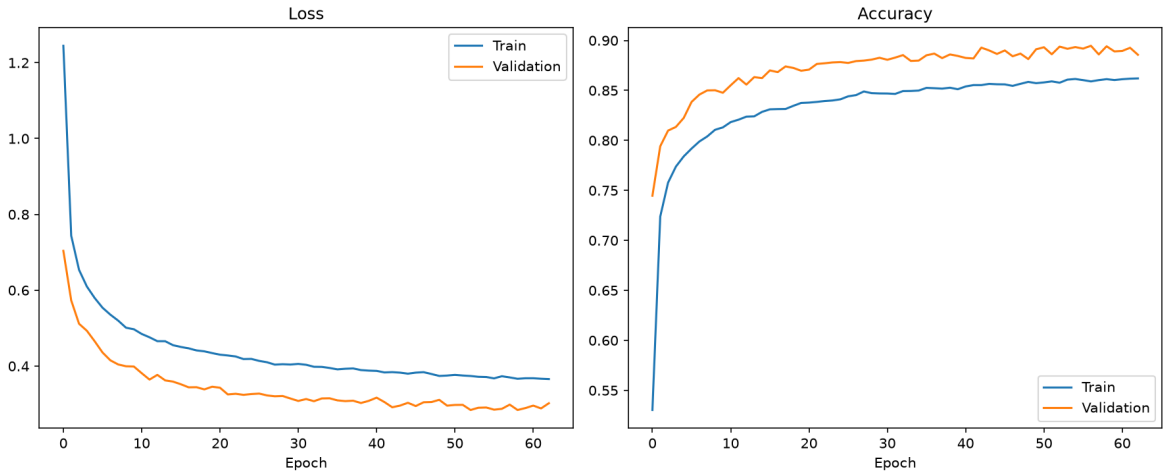


Figure 15: GRU training and validation curves. The selected epoch-53 checkpoint has validation loss 0.2842.

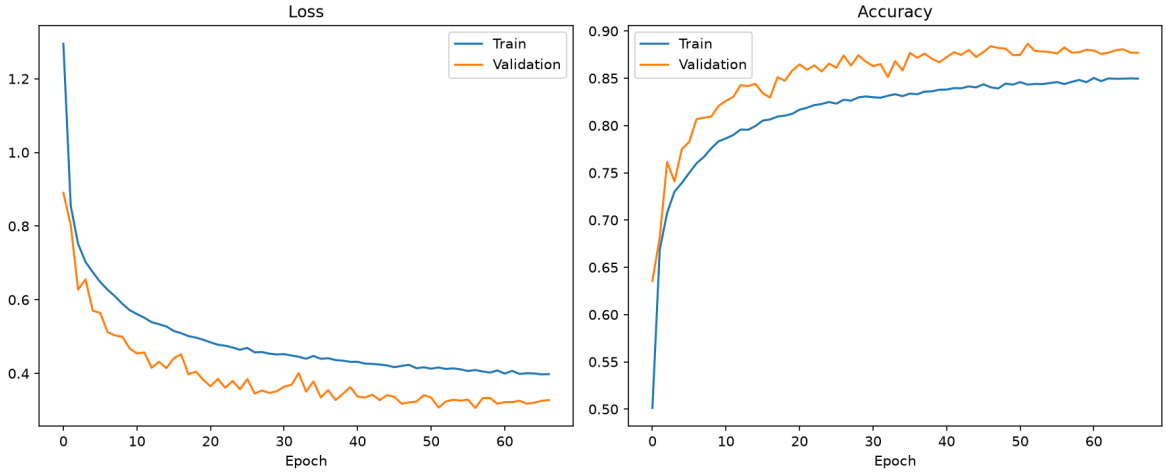


Figure 16: ViT training and validation curves. The minimum validation loss, 0.3062, occurs at epoch 57; training stopped after epoch 67.

Validation accuracy is often higher than training accuracy. This is expected because training accuracy is measured while random augmentation and dropout are active, whereas validation uses clean inputs and evaluation mode. The curves do not show a sustained validation-loss explosion; instead, early stopping prevents later fluctuations from replacing the best checkpoint. CNN converges to the strongest validation result, while Linear reaches a clear representational plateau.

ViT training and validation losses both fall sharply during early epochs and continue to trend downward more gradually. Validation loss remains below training loss, consistent with augmentation and dropout being active only during training. The epoch-57 checkpoint is retained by the minimum-validation-loss rule; ten subsequent epochs without a qualifying improvement trigger early stopping at epoch 67. Validation accuracy fluctuates around 0.88 near the end of training.

11 Quantitative Results

All reported values in Tables 10 and 11 come from the single augmented runs recorded in `Assignment1/outputs/log.txt` with seed 42. Metrics are computed on the official 10,000-image test set after restoring the minimum-validation-

loss checkpoint. The log includes complete predictive and resource results for all five architectures.

Table 10: Predictive performance on the Fashion-MNIST test set.

Method	Test loss	Accuracy	Macro precision	Macro recall	Macro-F1
Linear	0.7660	0.7378	0.7305	0.7378	0.7286
MLP	0.3756	0.8580	0.8564	0.8580	0.8560
CNN	**0.2069**	**0.9255**	**0.9272**	**0.9255**	**0.9259**
GRU	0.3130	0.8836	0.8829	0.8836	0.8821
ViT	0.3425	0.8793	0.8804	0.8793	0.8771

Table 11: Resource and efficiency measurements.

Method	Parameters	Model size	Train time	Inference	FLOPs/sample
Linear	7,850	0.030 MB	762.21 s	0.000822 ms	0.000016 B
MLP	567,434	2.165 MB	1,130.64 s	0.018203 ms	0.001133 B
CNN	1,691,914	6.454 MB	2,025.71 s	0.044741 ms	0.011022 B
GRU	61,962	0.236 MB	1,316.61 s	0.038619 ms	0.003450 B
ViT	803,338	3.064 MB	7,795.84 s	0.264764 ms	0.053043 B

CNN is the most accurate and has the best macro-F1, while Linear is smallest and fastest. GRU is the most parameter-efficient nonlinear model: it exceeds MLP by 2.56 accuracy points with 89.1% fewer parameters. ViT reaches 87.93% accuracy and 0.8771 macro-F1, slightly below GRU, while using about 13 times as many parameters and taking roughly 5.9 times as long to train. ViT inference is about 6.9 times slower than GRU (0.264764 ms/sample) and has the highest FLOPs (0.053043 billion/sample). The strong accuracy but high compute cost illustrates that global attention does not automatically improve this small-image task. Values are single-run measurements rather than mean $\pm$ standard deviation, and timing comparisons are subject to the hardware-recording limitation described in Table 8.

12 Qualitative Results

Figure 17 compares predictions from the restored Linear, MLP, CNN, GRU, and ViT checkpoints on the same Fashion-MNIST test examples. The selected rows include examples all models classify correctly, cases where structured models recover an example missed by a baseline, cases where CNN is correct but another model is not, and examples all models misclassify.

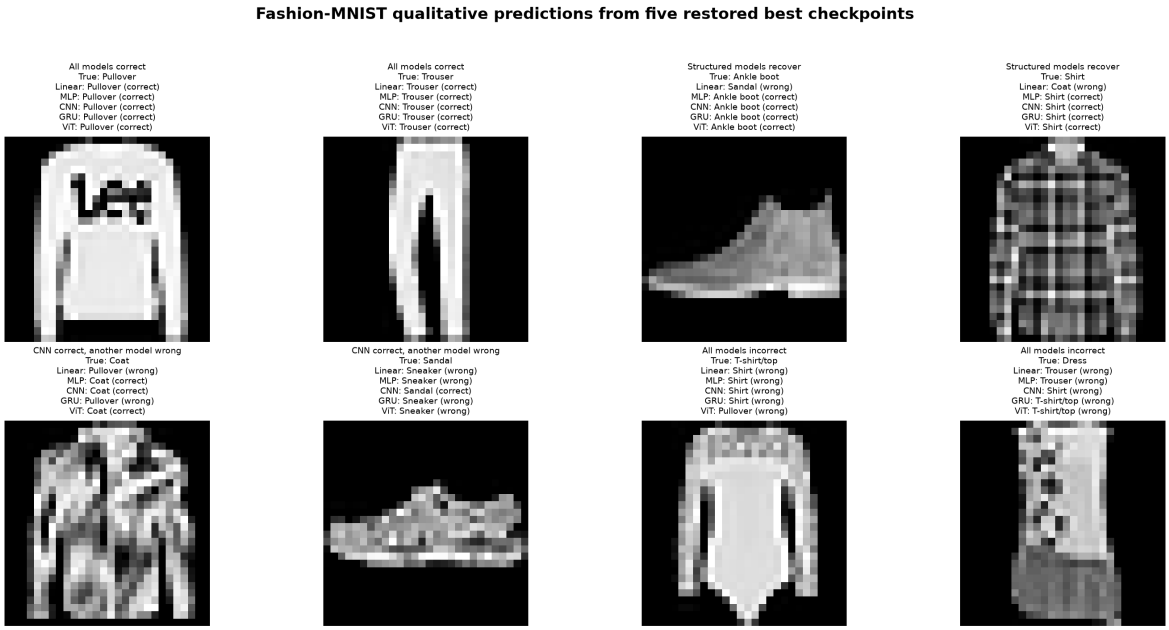


Figure 17: Qualitative test predictions from Linear, MLP, CNN, GRU, and ViT. Each example shows the true class and each model's predicted class.

The grid makes it possible to compare model behavior on individual images: some examples are straightforward for every model, while others separate the models or remain difficult for all five. These examples illustrate prediction behavior; the confusion matrices provide the broader class-level error summary.

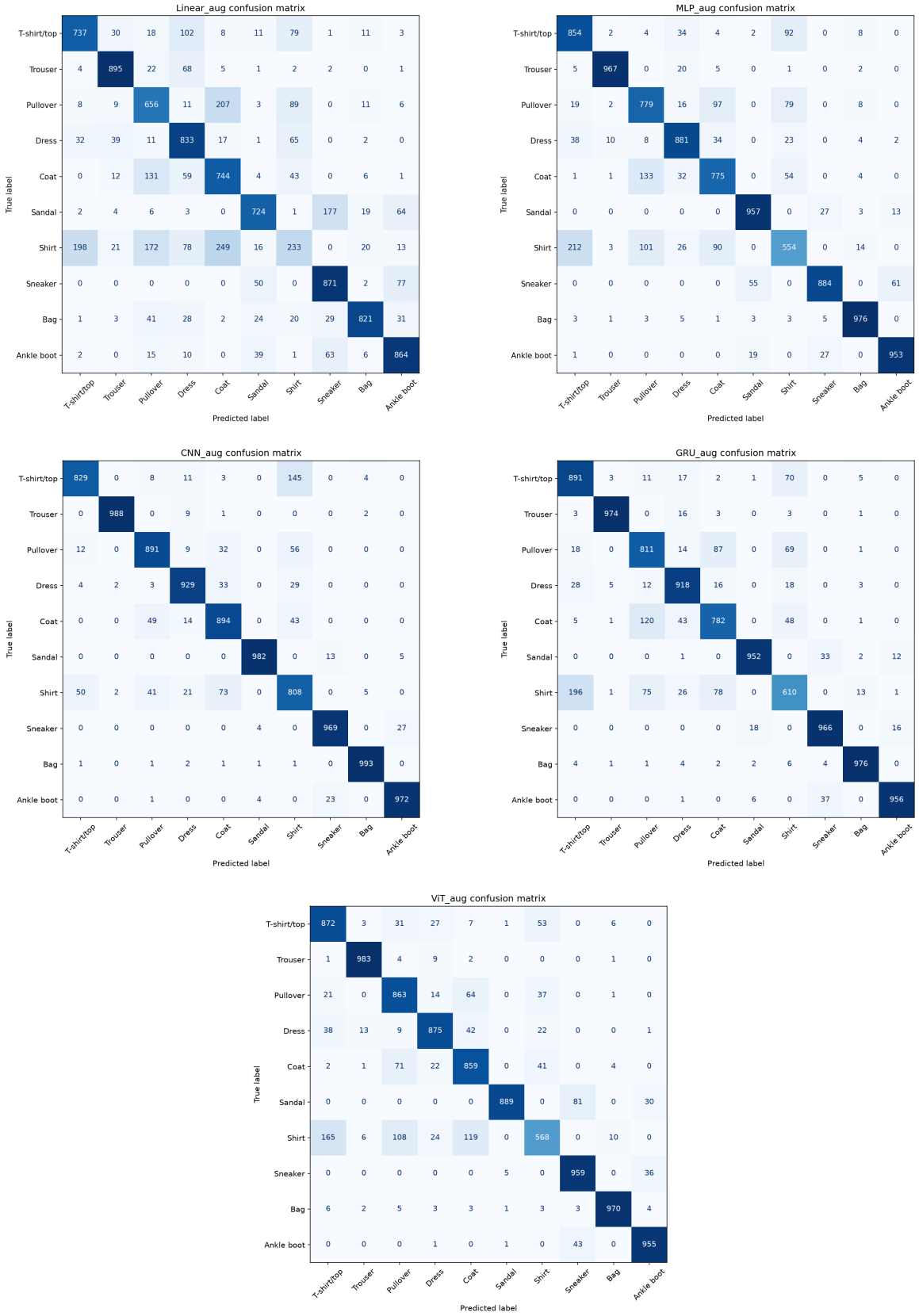


Figure 18: Test confusion matrices: Linear, MLP, CNN, GRU, and ViT.

13 Comparison and Discussion

Linear / softmax. The baseline reaches 73.78% accuracy and 0.7286 macro-F1 with only 7,850 parameters. Its additive pixel scoring is computationally efficient but cannot express nonlinear interactions or reusable local patterns. The very low Shirt F1 (0.3040) is direct evidence of under-capacity for overlapping garment categories.

MLP. Adding nonlinear dense layers raises accuracy by 12.02 percentage points and macro-F1 by 0.1274 over Linear. This supports the hypothesis that nonlinear combinations of pixels are necessary. The cost is 567,434 parameters and slower inference, while the flattened representation still has no explicit locality or translation prior.

GRU. Row-sequence processing reaches 88.36% accuracy and 0.8821 macro-F1, improving on MLP by 2.56 accuracy points and 0.0261 macro-F1 while using about one-ninth as many parameters. Shared recurrent weights encode dependencies between image rows efficiently. However, sequential computation makes GRU training 1.16 times and inference 2.12 times as slow as MLP in the recorded run. Its top-to-bottom ordering also treats horizontal and vertical relationships asymmetrically.

CNN. CNN ranks first at 92.55% accuracy and 0.9259 macro-F1. Local 3×3 filters, shared weights, and pooling match the two-dimensional image structure and improve Shirt F1 to 0.7762. Relative to GRU, CNN gains 4.19 accuracy points and 0.0438 macro-F1, but uses 27.3 times as many parameters and takes 1.54 times as long to train.

Vision Transformer. ViT reaches 87.93% accuracy and 0.8771 macro-F1, placing it below GRU and CNN but above MLP and Linear on both measures. It uses 803,338 parameters, which is about 13 times GRU's count, and takes 7,795.84 seconds to train. Its inference time is 0.264764 ms/sample, the highest recorded, and its Shirt recall is only 0.568. This result does not support the hypothesis that global patch attention would outperform the recurrent sequence model or CNN under this configuration. Its positional embeddings retain patch order, but the model has less built-in locality and a substantial compute cost for 28-by-28 images.

Across all five models, the results support the value of nonlinear representations over a single linear boundary, but they do not show that increasing architectural complexity always improves accuracy. CNN gives the best predictive performance; Linear is the smallest and fastest; GRU offers a strong accuracy-to-parameter trade-off; and ViT is slower and larger than GRU while scoring slightly lower. Hardware details were not recorded precisely, so runtime comparisons are indicative rather than controlled benchmarks. The single-seed design also limits conclusions about run-to-run variability.

14 Error Analysis

The confusion matrices localize the main errors in visually similar categories. Table 12 reports error counts derived from the class recalls in the 1,000-example-per-class test set.

Table 12: Major error groups across all models. Counts are errors within the indicated true-class subset.

Error group	Errors: Linear / MLP / CNN / GRU / ViT	Cause and improvement direction
Shirt	767 / 446 / 192 / 390 / 432 out of 1,000	Shirt shares outlines and textures with T-shirt/top, Pullover, and Coat. For ViT, the confusion matrix shows 165 Shirts predicted as T-shirt/top, 119 as Coat, and 108 as Pullover. Use class-aware hard-example sampling, stronger local features, and inspection of ambiguous annotations.
Pullover + Coat	600 / 446 / 215 / 407 / 278 out of 2,000	Similar sleeves, torso shapes, and low grayscale resolution. ViT confuses Pullover with Coat 64 times and Coat with Pullover 71 times. Test deeper convolutional features or targeted augmentation that preserves garment identity.
Sandal + Sneaker + Ankle boot	541 / 206 / 77 / 126 / 197 out of 3,000	Pose and sole shape overlap. ViT confuses Sandal with Sneaker 81 times and Ankle boot with Sneaker 43 times. Analyze high-confidence footwear mistakes and test targeted spatial features.

For GRU, the most prominent single confusion is Shirt predicted as T-shirt/top (196 cases); Pullover and Coat are also frequently exchanged. For ViT, Shirt is the weakest class, with 432 errors out of 1,000; its mistakes are spread mainly across T-shirt/top, Pullover, and Coat. CNN has the fewest errors in the listed clothing and footwear groups. These patterns are consistent with overlap among low-resolution garment silhouettes; architecture changes alone may not resolve every ambiguous case.

15 Limitations and Conclusion

Data limitations. Fashion-MNIST contains centered, low-resolution grayscale objects with balanced labels. Results may not transfer to color images, cluttered backgrounds, larger resolutions, or naturally imbalanced data. Several upper-body categories are intrinsically ambiguous at 28×28 resolution.

Method limitations. All five architectures were implemented and evaluated, but the configurations were fixed rather than exhaustively tuned and model capacities are not matched. No formal augmentation, dropout, representation, or parameter-budget ablation was run. ViT's four-layer encoder and 50-token sequence are computationally expensive for this small input, while the model's learned global attention does not encode locality as strongly as a CNN.

Experimental limitations. Results come from one deterministic seed, so run-to-run variance is unknown. The exact CPU model and complete session identity were not recorded; therefore, training and inference measurements indicate practical cost but are not hardware-controlled benchmarks. The qualitative set is deliberately small and illustrates behavior rather than estimating frequency.

Expected generalization and future work. CNN's advantage is observed on this dataset and configuration; its magnitude and ranking should be confirmed across seeds and datasets. Future work is to repeat experiments across multiple seeds, log exact hardware and software automatically.

Conclusion. Under the shared Fashion-MNIST pipeline, architectural inductive bias materially changes accuracy and efficiency across all five models. Linear is the cheapest but underfits; MLP gains from nonlinear capacity; GRU provides a strong accuracy–parameter compromise; and CNN is the strongest measured classifier, reaching 92.55% accuracy and 0.9259 macro-F1. ViT reaches 87.93% accuracy and 0.8771 macro-F1 but has the highest measured inference time and FLOPs. Thus, global patch attention does not outperform the convolutional or row-recurrent alternatives in this experiment. CNN is preferred when predictive quality is primary, Linear when minimal size and latency matter, and GRU when parameter efficiency matters. These conclusions are limited to one seed and incompletely characterized CPU hardware.

AI Usage Disclosure

Course: CO3133 — Deep Learning and Its Applications · Semester-261
Group: G-10
Assignment: Assignment 1 — Foundations of Deep Learning Pipelines and Architectures
This file logs all generative AI tool usage for Assignment 1.

Detailed Log

```
Tool: Gemini
Used by: Trinh Le Minh
Task: Implementing model checkpointing and analyzing loss curves
Prompt summary: Asked for the definition and code implementation of machine learning
    checkpoints, and requested an analysis of a loss/accuracy graph to determine if the
     model was overfitting.
AI contribution: Provided the PyTorch code logic to save and load best-model
    checkpoints based on validation loss. Analyzed the provided training/validation
    graphs, confirming healthy training and explaining how the MLP's Dropout and Data
    Augmentation caused validation metrics to initially outperform training metrics.
Student verification: Verified the health of the model's training/validation loss curve
     to confirm no overfitting was occurring; reviewed and integrated the PyTorch
    checkpointing code logic into the training loop.
Affected files/sections: trainer.py
Responsible member: Trinh Le Minh

Tool: OpenAI Codex (GPT-5)
Used by: Trinh Le Minh
Task: Building and debugging a reproducible FashionMNIST training and evaluation
    pipeline
Prompt summary: Asked for guidance and implementation support for EDA, train/validation
    /test splitting, preprocessing and augmentation, Linear/MLP training, evaluation
    metrics, checkpointing, and reproducibility. Later reported a PyTorch checkpoint-
    loading error caused by NumPy scalar metadata.
AI contribution: Connected the EDA and metrics modules to the main workflow; added
    saved EDA figures, classification metrics, confusion matrices, best-validation-loss
     checkpoints, deterministic seeding, DataLoader worker seeding, experiment metadata
    , and model fingerprints. Diagnosed the checkpoint error, converted checkpoint
    metadata to native Python types, and added a restricted compatibility loader for
    the existing checkpoint.
Student verification: Ran the MLP training workflow and supplied the resulting
    checkpoint-loading traceback; inspected the generated metric files and checkpoints.
     Existing best_mlp.pt was subsequently loaded successfully, and syntax/static
    checks passed.
Affected files/sections: Assignment1/code/main.py, config.py, dataloader.py, edaworker.
    py, metrics.py, trainer.py, plotter.py; Assignment1/code/README.md; Assignment1/
    checkpoints/README.md.
Responsible member: Trinh Le Minh

Tool: OpenAI Codex (GPT-5)
Used by: Trinh Le Minh
Task: Explaining and documenting the CNN and GRU architectures
Prompt summary: Asked for help explaining how the CNN preserves spatial structure, how
    the GRU treats each image as a sequence of rows, why the GRU acts as a sequence
    encoder, and how its 60,672 recurrent parameters are calculated.
```

```
AI contribution: Explained the architectural inductive biases and tensor
    transformations of both models; derived the GRU parameter count from its input-to-
    hidden weights, hidden-to-hidden weights, and two bias vectors; and helped prepare
    the CNN and GRU architecture descriptions and diagrams used on the Assignment 1
    page.
Student verification: Cross-checked every layer, tensor shape, and parameter count
    against models.py and verified the reported CNN and GRU metrics against outputs/log
    .txt and the generated result figures.
Affected files/sections: Assignment1/index.html, Assignment1/diagram-models/cnn-model.
    svg, Assignment1/diagram-models/gru-model.svg; CNN and GRU methodology and
    inductive-bias sections.
Responsible member: Trinh Le Minh

Tool: OpenAI Codex (GPT-5)
Used by: Trinh Le Minh
Task: Completing the Assignment 1 LaTeX report from verified project evidence
Prompt summary: Asked to use the course handbook, assignment page, source code, logs,
    checkpoints, diagrams, and generated outputs to complete the existing report
    without changing its structure.
AI contribution: Replaced report placeholders and outdated metrics; integrated the
    pipeline and model diagrams, four-model methodology and results, learning curves,
    confusion matrices, qualitative checkpoint predictions, error counts, limitations,
    references, and evidence-based conclusion. Explicitly identified the missing
    Transformer and formal ablation rather than inventing results.
Student verification: Compared all numeric claims with outputs/log.txt, checked
    architecture details against models.py, visually inspected generated figures, and
    compiled the LaTeX report to identify formatting and reference errors.
Affected files/sections: report/report-assignment-1/; Assignment1/code/qualitative.py;
    Assignment1/outputs/metrics/qualitative_predictions.png.
Responsible member: Trinh Le Minh

Tool: GPT (via VSCode)
Used by: Tran Phuoc Sang
Task: Coding assistance for metrics and plotter files — task-specific EDA
Prompt summary: Asked for help implementing metric computation and EDA plot functions
AI contribution: Suggested structure for metric aggregation and matplotlib plot helpers
Student verification: Ran outputs against expected values; visually verified EDA plots
Affected files/sections: Assignment1/code/metrics.py; Assignment1/code/edaworker.py
Responsible member: Tran Phuoc Sang

Tool: Claude Sonnet 4.6 (Antigravity IDE)
Used by: Tran Phuoc Sang
Task: LaTeX report writing assistance
Prompt summary: Asked for help drafting methodology and results sections in LaTeX
AI contribution: Suggested LaTeX structure, table formatting, phrasing for Sec. -34
Student verification: Cross-checked all claims against actual experiment outputs and
    code
Affected files/sections: report/report-assignment-1/ (Methodology, Results sections)
Responsible member: Tran Phuoc Sang

Tool: OpenAI Codex (GPT-6)
Used by: Trinh Le Minh
Task: Learning the Vision Transformer and understanding how its code follows the common
     Transformer architecture
Prompt summary: Asked for teaching-oriented explanations of the ViT implementation,
    including a detailed walkthrough of its forward pass and how the standard
    Transformer ideas apply to image patches.
```

```
AI contribution: Helped implement the initial ViTransformer structure, then explained
    patch embeddings, positional embeddings, the class token, self-attention, the feed-
    forward block, residual connections, normalization, and classification; connected
    these ideas to the code and its forward pass.
Student work and verification: Studied the explanations, worked through and reviewed
    the implementation, integrated the model into the project workflow, and ran it.
    Checked the architecture and tensor dimensions against the code, and checked
    evaluation claims against outputs/log.txt and saved figures.
Affected files/sections: Assignment1/code/models.py; ViT methodology and results in
    Assignment1/index.html and report/report-assignment-1/.
Responsible member: Trinh Le Minh
```

Student Responsibility Acknowledgement

☒ All members understand and can explain every part of the submission.

☒ No unchecked AI-generated code or text was submitted as-is.

☒ AI was not used to fabricate data, experimental results, citations, or references.

☒ No experiments were claimed that were not actually run.

☒ No private/restricted/credential-bearing data was pasted into AI tools.

☒ All AI-assisted content was verified via source code, real runs, official docs, or scholarly sources.
